

Object-Oriented Programming

Lecturer: NGUYEN Thi Thu Trang, trangntt@soict.hust.edu.vn

Lab 01: Java Basics

In this lab, you will practice with:

- Setting up your GitHub repository for this course and basic Git commands
- Setting up the Java development environment (JDK)
- Writing and compiling small Java programs using the JDK's command-line tools
- Java language basics: identifiers, data types, casting, operators, control statements, and arrays

Keywords: Git, GitHub, JDK, JRE, Java installation, identifiers, data types, casting, control statements, arrays

0 Submission Guidelines

You are required to submit all your work by pushing to your GitHub repository (see Repository Setup in section 1). For this lab, you have to turn in:

- Source code of all exercises (3.2.1 – 3.2.4, 4.1 – 4.3, 5.1 & 5.2), inside the Lab01/ directory.
- A screenshot (PNG or JPG) of the console output for each exercise, inside Lab01/screenshots/.

Deadline: Please refer to the assignment post on the class channel.

1 Repository Setup

You will use a single GitHub repository for all labs in this course. Set it up once now, following the steps below.

1.1 Create your GitHub Repository

- Log in to GitHub (create a free account if you don't have one yet).
- Click the "+" icon in the top-right corner of the page, then choose "New repository".

In the Repository name field, enter a name following this convention:

IT3101E.176269.20261.<StudentID>.<FullNameNoSpaceNoAccent>

FullNameNoSpaceNoAccent means your full name written with no spaces and no diacritics, all lowercase except the first letter of each word (e.g., HoVaTen). For example:

IT3101E.176269.20261.202610000.NguyenVanA

- Under Visibility, choose Private, then click "Create repository".

- Go to Settings → Collaborators, and add the following accounts so the lecturer and TA can review your work:
 - trangntt.for.student@gmail.com (Lecturer)
 - ykhanh205.1@gmail.com (TA)
 - pvanh780@gmail.com (TA)

Note: Keep using this same repository for every lab this semester - do not create a new one each week.

1.2 Git Basics

Git is a free, open-source distributed source-control system that lets you work with local and remote repositories. Install it on your machine first, using the instructions for your operating system below.

Windows:

- Download Git for Windows (which includes Git Bash, a command-line interface for Git) from:
<https://git-scm.com/download/win>
- Run the installer - the default options work fine ("Next, Next, Next" does the trick).

Linux:

```
$ sudo apt-get install git
```

macOS:

```
$ git --version
```

If Git is not installed, macOS will prompt you to install it.

To verify the installation, open a Command Prompt/Terminal (or Git Bash on Windows) and run:

```
$ git --version
```

For a more detailed guide, see [this Atlassian tutorial on setting up a repository](#).

Once Git is installed, use the following basic commands to save and submit your work. (More advanced workflows, such as branching and pull requests, will be introduced in a later lab, once you have more than one piece of work to manage at a time.)

Clone your repository to your local machine:

```
$ git clone <your repository URL>
```

After making changes, stage, commit, and push them:

```
$ git add .
$ git commit -m "Complete assignment 3.2.1"
$ git push origin master
```

Note: Every exercise's submission steps in this lab will simply refer back to these three commands.

2 Getting Started

2.1 Java Development Kit

Java Platform, Standard Edition Development Kit (JDK) is a development environment for building applications, applets, and components using the Java programming language. There are many releases of the platform that are available to download. Among them, JDK 8 is the most widely-used version of Java and it is also the last long-term support (LTS) release that contains JavaFX (which we will be working extensively with later on in this course). For the above reasons, it is required that JDK 8 is installed for all the labs in this course. However, if you have installed a later version, you can still install JavaFX separately (there will be an installation guide in the JavaFX lab in this case).

The installation steps for JDK 8 are illustrated as follows:

Step 1: Check if JDK has been pre-installed

1. Open a Command Prompt (on Windows) or a Terminal (on Linux or macOS).
2. Issue the following command.

```
$ javac -version
```

In case a JDK version number is returned (e.g., JDK x.x.x), then JDK has already been installed. When the JDK version is prior to 1.8, or a message such as "'javac' is not recognized..." is shown, proceed to Step 2 to install Oracle JDK 8.

Note: Linux usually chooses OpenJDK as its default JDK since OpenJDK is open source. However, Oracle JDK is not completely compatible with OpenJDK and it is recommended to use Oracle JDK.

Step 2: Download Oracle JDK 8

Go to the Java SE Development Kit (JDK) 8 download site, then download the installation file for your operating system. You may need an Oracle Account to download, as the Oracle JDK License has changed for releases since April 16, 2019.

Step 3: Install and Configure

Windows:

1. Install Oracle JDK 8. Run the downloaded installer and follow the instructions.
2. Configure. Launch Control Panel → System and Security → System → Advanced system settings → Environment Variables. In the lower list "System variables", look for two variables:
 - JAVA_HOME: set its value to the path of the JDK installation folder, e.g. C:\Program Files\Java\jdk1.8.0_241.
 - Path: add the following entry to it: %JAVA_HOME%\bin

Linux:

1. Create installation directory.

```
$ cd /usr/local  
$ sudo mkdir java
```

2. Extract the downloaded package to the installation directory.

```
$ cd /usr/local/java
$ sudo tar xzvf ~/Downloads/jdk-8u241-linux-x64.tar.gz
```

3. Inform Linux to use this JDK/JRE.

```
$ sudo update-alternatives --install "/usr/bin/java" "java" \
"/usr/local/java/jdk1.8.0_241/bin/java" 1
$ sudo update-alternatives --set java /usr/local/java/jdk1.8.0_241/bin/java
```

macOS: Double-click the DMG file and follow the instructions.

Step 4: Verify the JDK Installation

Issue the following command.

```
$ javac -version
```

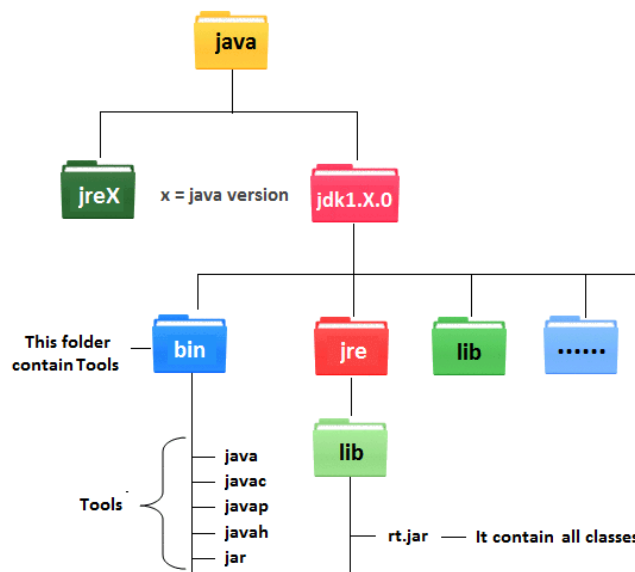


Figure 1. Development Environment.

2.2 Programming Text Editor

For the next section, you can use any text editor to write your Java source code. Here, the Notepad text editor is used as an illustration.

3 First Programs

3.1 Java Programming Steps

The steps in writing a Java program are illustrated as follows.

Step 1: Write the source code and save it in, e.g., "HelloWorld.java" file.

Step 2: Compile the source code into Java portable bytecode using the JDK's Java compiler by issuing the following command.

```
$ javac HelloWorld.java
```

Step 3: Run the compiled bytecode using the JDK's Java Runtime by issuing the following command.

```
$ java HelloWorld
```

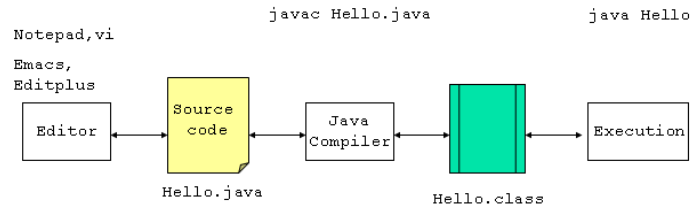


Figure 2. Compile a Java application by command line

```

HelloWorld.java:
    public class HelloWorld {
        public static void main(String[] args) {
            System.out.println("Hello, world");
        }
    }

```

Annotations for the code above:

- same name with the class* points to `HelloWorld.java:`
- this is a class* points to `public class`
- class name* points to `HelloWorld`
- start of the class* points to the opening curly brace `{`
- method name* points to `main`
- a statement it says print to standard output* points to `System.out.println("Hello, world");`
- end of the class* points to the closing curly brace `}`
- public, so that everyone can access* points to `public`

Figure 3. The first Java application

The result is shown in Figure 4.

```
%> javac HelloWorld.java
%> java HelloWorld
Hello, world
```

Figure 4. Result of the first Java application

3.2 The Very First Java Programs

3.2.1 Write, compile the first Java application

Step 1: Create a new file. From the Notepad interface, choose File → New File.

Step 2: Save the file with the name "HelloWorld.java".

Step 3: Write the source code, shown in Figure 5.

```

1 //Example 1: HelloWorld.java
2 //Text-printing program
3 public class HelloWorld {
4
5     public static void main(String args[]){
6         System.out.println("Xin chao \n cac ban!");
7         System.out.println("Hello \t world!");
8
9     } // end of method main
10 }

```

Figure 5. The First Java Application

Step 4: Compile and run.

```

$ javac HelloWorld.java
$ java HelloWorld

```

Step 5: After completing this assignment, add and commit your work: "git add ." then 'git commit -m "Complete assignment 3.2.1"'.

3.2.2 Write, compile the first dialog Java program

Save the file as "FirstDialog.java". The source code is shown in Figure 6.

```

1 // Example 2: FirstDialog.java
2 import javax.swing.JOptionPane;
3 public class FirstDialog{
4     public static void main(String[] args){
5         JOptionPane.showMessageDialog(null,"Hello world! How are you?");
6         System.exit(0);
7     }
8 }

```

Figure 6. The First Dialog Java Application

```

$ javac FirstDialog.java
$ java FirstDialog

```

As in step 5 of section 3.2.1, add and commit your work.

3.2.3 Write, compile the first input dialog Java application

Save the file as "HelloNameDialog.java". The source code is shown in Figure 7.

```

1 // Example 3: HelloNameDialog.java
2 import javax.swing.JOptionPane;
3 public class HelloNameDialog{
4     public static void main(String[] args){
5         String result;
6         result = JOptionPane.showInputDialog("Please enter your name:");
7         JOptionPane.showMessageDialog(null, "Hi " + result + "!");
8         System.exit(0);
9     }
10 }

```

Figure 7. The First Input Dialog Java Application

```

$ javac HelloNameDialog.java
$ java HelloNameDialog

```

As in step 5 of section 3.2.1, add and commit your work.

3.2.4 Write, compile, and run the following example

Save the file as "ShowTwoNumbers.java". The source code is shown in Figure 8.

```
1 // Example 5: ShowTwoNumbers.java
2 import javax.swing.JOptionPane;
3 public class ShowTwoNumbers {
4     public static void main(String[] args){
5         String strNum1, strNum2;
6         String strNotification = "You've just entered: ";
7
8         strNum1 = JOptionPane.showInputDialog(null,
9             "Please input the first number: ", "Input the first number",
10            JOptionPane.INFORMATION_MESSAGE);
11         strNotification += strNum1 + " and ";
12
13         strNum2 = JOptionPane.showInputDialog(null,
14             "Please input the second number: ", "Input the second number",
15            JOptionPane.INFORMATION_MESSAGE);
16         strNotification += strNum2;
17
18         JOptionPane.showMessageDialog(null, strNotification,
19             "Show two numbers", JOptionPane.INFORMATION_MESSAGE);
20         System.exit(0);
21     }
22 }
```

Figure 8. Java Application showing two entered numbers and their sum

```
$ javac ShowTwoNumbers.java
$ java ShowTwoNumbers
```

As in step 5 of section 3.2.1, add and commit your work.

4 Java Language Basics

This section walks through the core building blocks of the Java language: identifiers, data types, operators, control statements, and arrays. Read each short recap, then complete the exercise that follows before moving to the next topic.

4.1 Identifiers, Data Types and Casting

- Identifiers name variables, methods, and classes. They may contain letters, digits, "\$" or "_", but must not start with a digit or match a keyword. Java is case-sensitive.
- Java has two categories of data types: primitive (int, double, char, boolean, ...) and reference (array, object).
- Casting converts a value from one data type to another. Widening (e.g., int → double) happens automatically; narrowing (e.g., double → int) must be done explicitly and may lose information.

Exercise 4.1: DeclareVariables.java

Create a new file named "DeclareVariables.java" with the following content, then compile and run it.

```

1 public class DeclareVariables {
2     public static void main(String[] args) {
3         String title = "The Lion King";
4         double cost = 19.95;
5         int id = 1;
6         boolean isAvailable = true;
7
8         System.out.println(title);
9         System.out.println(cost);
10        System.out.println(id);
11        System.out.println(isAvailable);
12
13        // Narrowing cast: double -> int
14        int roundedCost = (int) cost;
15        System.out.println(roundedCost);
16    }
17 }
18

```

Question: What value is printed for roundedCost? Is it rounded to the nearest integer, or truncated? Explain why.

Don't forget to add and commit your work using 'git add .' and 'git commit -m <message>'.

4.2 Control Statements

- if-else: executes one of two blocks of statements depending on whether a boolean condition is true or false.

Exercise 4.2a: MediaDiscount.java (if-else)

Create a new file named "MediaDiscount.java" that reads the cost of a media item and prints "Discounted" if the cost is over 20, or "Full price" otherwise.

```

1 import javax.swing.JOptionPane;
2
3 public class MediaDiscount {
4     public static void main(String[] args) {
5         String strCost = JOptionPane.showInputDialog("Enter the cost of the media:");
6         double cost = Double.parseDouble(strCost);
7
8         if (cost > 20) {
9             System.out.println("Discounted");
10        } else {
11            System.out.println("Full price");
12        }
13    }
14 }

```

- switch-case: checks a single variable against several possible values and executes the matching case.

Exercise 4.2b: MediaTypeName.java (switch-case)

Create a new file named "MediaTypeName.java" that asks the user to choose 1, 2 or 3, and prints the corresponding media type name (DVD, CD, Book).

```
1 import javax.swing.JOptionPane;
2
3 public class MediaTypeName {
4     public static void main(String[] args) {
5         String strOption = JOptionPane.showInputDialog(
6             "Choose a media type (1: DVD, 2: CD, 3: Book):");
7         int option = Integer.parseInt(strOption);
8         String mediaType;
9
10        switch (option) {
11            case 1:
12                mediaType = "DVD";
13                break;
14            case 2:
15                mediaType = "CD";
16                break;
17            case 3:
18                mediaType = "Book";
19                break;
20            default:
21                mediaType = "Unknown";
22        }
23
24        System.out.println("You selected: " + mediaType);
25    }
26 }
```

Don't forget to add and commit your work using 'git add .' and 'git commit -m <message>'.

4.3 Arrays

- An array is a fixed-size collection of elements of the same type, indexed starting from 0.
- An array can be declared and initialized at the same time, e.g. `double[] cost = {19.95, 24.95};`

Exercise 4.3: PrintCostArray.java

Create a new file named "PrintCostArray.java" that declares an array of costs and prints each element, followed by the number of items in the array.

```

1 public class PrintCostArray {
2     public static void main(String[] args) {
3         double[] cost = {19.95, 24.95, 18.99, 15.50, 22.00};
4
5         for (int i = 0; i < cost.length; i++) {
6             System.out.println(cost[i]);
7         }
8
9         System.out.println("Number of items: " + cost.length);
10    }
11 }

```

Don't forget to add and commit your work using 'git add .' and 'git commit -m <message>'.

5 Applying Operators and Control Statements

5.1 Write a program to calculate sum, difference, product, and quotient of 2 double numbers which are entered by users.

Notes: To convert from String to double, you can use `double num1 = Double.parseDouble(strNum1)`. Check the divisor of the division.

Don't forget to add and commit your work using 'git add .' and 'git commit -m <message>'.

5.2 Write a program to solve:

For simplicity, we only consider the real roots of the equations in this task.

The first-degree equation (linear equation) with one variable

A first-degree equation with one variable can have a form such as $ax + b = 0$ ($a \neq 0$). You should handle the case where the user inputs value 0 for a .

The system of first-degree equations (linear system) with two variables

A system of first-degree equations with two variables x_1 and x_2 can be written as $a_{11}x_1 + a_{12}x_2 = b_1$ and $a_{21}x_1 + a_{22}x_2 = b_2$. You should handle the case where the values of the coefficients produce infinitely many solutions, and the case where they produce no solution.

$$D = \begin{vmatrix} a_{11} & a_{12} & a_{21} & a_{22} \end{vmatrix} = a_{11}a_{22} - a_{21}a_{12} \quad D_1 = \begin{vmatrix} b_1 & a_{12} & b_2 & a_{22} \end{vmatrix} = b_1a_{22} - b_2a_{12} \quad D_2 = \begin{vmatrix} a_{11} & b_1 & a_{21} & b_2 \end{vmatrix} = a_{11}b_2 - a_{21}b_1$$

Hint: use the determinants shown above.

The second-degree equation with one variable

A second-degree equation with one variable (i.e., quadratic equation) can have a form such as $ax^2 + bx + c = 0$, where $a \neq 0$. You should handle the case where the coefficients produce a double root, the case where they produce no root, and the case where the user inputs value 0 for a . Hint: use the discriminant $\Delta = b^2 - 4ac$.

After completing the code in sections 4 and 5, push all of your changes to the "master" branch of the valid repository you have created.

Hint: Use "git push origin <name of the branch that you want to push>", in this case "master", to push all of your work to the master branch.